

UNIVERSIDAD PERUANA LOS ANDES



CURSO: DISEÑO DE EXPERIENCIA DE USUARIO UX&UI

***DOCENTE: Raul Enrique
Fernandez Bejarano***

ALUMNO:

Ñahuincopa Huamani Lian Renzo

CICLO: VI

Semana 12

***HUANCAYO-
2025***

Estrategia de Backup completo, diferente, transaccional

1. Estrategias de Backup

Backup Completo (Full Backup)

Copia íntegra de toda la base de datos, incluyendo todos los objetos, datos y estructura. Captura el estado completo de la base de datos en un momento específico. Es la base fundamental para cualquier estrategia de recuperación y permite restaurar la base de datos completamente por sí solo.

Backup Diferencial

Copia únicamente los datos que han cambiado desde el último backup completo. Acumula todos los cambios realizados desde el último full backup, por lo que cada backup diferencial es progresivamente más grande. Para restaurar, se necesita el último backup completo más el último backup diferencial.

Backup Transaccional (Transaction Log Backup)

Copia el registro de transacciones (transaction log), capturando todas las transacciones confirmadas desde el último backup de log. Permite recuperar puntos en el tiempo y minimiza la pérdida de datos. Solo disponible en modelos de recuperación FULL o BULK_LOGGED. Para restaurar completamente se requiere: backup completo + backups diferenciales (si existen) + secuencia de backups transaccionales.

2. Restauración de Bases de Datos en Distintos Escenarios

Restauración completa

Recuperación total de la base de datos cuando no hay corrupción de datos ni pérdida parcial. Se restaura el último backup completo y se aplican los backups diferenciales y transaccionales necesarios.

Restauración ante fallo de hardware

Recuperación cuando se pierden archivos físicos (.mdf, .ldf). Requiere restaurar en una nueva ubicación o hardware, aplicando la secuencia completa de backups disponibles.

Restauración por corrupción de datos

Recuperación selectiva cuando hay corrupción en páginas específicas o tablas. Puede requerir restauración parcial o uso de técnicas avanzadas como RESTORE PAGE.

Restauración de emergencia

Proceso acelerado para poner operativa la base de datos lo más rápido posible, aplicando solo los backups esenciales y posponiendo recuperaciones completas.

3. Mantenimiento y Verificación de Copias de Seguridad

Verificación de integridad

Proceso de validar que los archivos de backup no están corruptos y pueden ser utilizados para restauración. Se utiliza **RESTORE VERIFYONLY** para comprobar sin restaurar realmente.

Pruebas de restauración

Simulacros periódicos de recuperación en ambientes de prueba para validar que los backups funcionan correctamente y que los tiempos de recuperación cumplen los objetivos (RTO - Recovery Time Objective).

Mantenimiento de archivos

Gestión del espacio de almacenamiento, rotación de backups antiguos, limpieza de archivos obsoletos según políticas de retención, y monitoreo del crecimiento de los archivos de backup.

Documentación

Registro detallado de ubicaciones de backups, fechas, tamaños, y procedimientos de restauración actualizados.

4. Uso de BACKUP DATABASE y RESTORE DATABASE

BACKUP DATABASE

Comando T-SQL para crear copias de seguridad de bases de datos. Sintaxis básica:

- **BACKUP DATABASE nombre TO DISK='ruta'** (backup completo)
- **BACKUP DATABASE nombre TO DISK='ruta' WITH DIFFERENTIAL** (diferencial)
- **BACKUP LOG nombre TO DISK='ruta'** (transaccional)

RESTORE DATABASE

Comando T-SQL para restaurar bases de datos desde archivos de backup. Sintaxis básica:

- **RESTORE DATABASE nombre FROM DISK='ruta' WITH NORECOVERY** (restauración en secuencia)
- **RESTORE DATABASE nombre FROM DISK='ruta' WITH RECOVERY** (restauración final)
- **RESTORE LOG nombre FROM DISK='ruta' WITH NORECOVERY** (aplicar logs)

Opciones importantes incluyen REPLACE (sobrescribir), MOVE (cambiar ubicación de archivos), STOPAT (restauración punto en el tiempo).

5. Planes de Mantenimiento y Políticas de Retención

Planes de Mantenimiento

Conjuntos automatizados de tareas programadas que incluyen backups, verificaciones de integridad, reorganización de índices, y limpieza de archivos. Se configuran mediante SQL Server Management Studio o scripts T-SQL con SQL Server Agent.

Políticas de Retención

Reglas que definen cuánto tiempo se conservan los backups antes de ser eliminados. Consideran:

- Requisitos legales y de cumplimiento normativo
- Espacio de almacenamiento disponible
- Objetivos de recuperación (RPO - Recovery Point Objective)
- Frecuencia de backups según criticidad del negocio

Ejemplo común: backups completos semanales conservados 4 semanas, diferenciales diarios conservados 7 días, transaccionales cada hora conservados 24 horas.

6. Restauración Punto en el Tiempo (Point-in-Time Recovery)

Capacidad de restaurar la base de datos a un momento específico en el pasado, no solo al momento del último backup. Permite recuperar datos hasta justo antes de un error (como eliminación accidental de registros).

Requisitos:

- Modelo de recuperación FULL
- Cadena completa de backups: completo + diferenciales + todos los logs transaccionales hasta el punto deseado

Proceso:

1. Restaurar backup completo con NORECOVERY
2. Restaurar backup diferencial (si existe) con NORECOVERY
3. Restaurar logs transaccionales secuencialmente con NORECOVERY
4. Aplicar último log con **STOPAT='fecha_hora'** y RECOVERY

Esto permite recuperación quirúrgica de datos minimizando pérdida de información y es esencial para cumplir con RPO bajos en entornos críticos.

PRÁCTICA DE LA SEMANA 12**1: ESTRATEGIA DE BACKUP COMPLETO DIARIO Y VERIFICACIÓN AUTOMÁTICA****ENUNCIADO DEL EJERCICIO**

Implementar un script que haga un backup FULL diario de la base QhatuPeru, almacene el archivo con fecha, y verifique la integridad del backup (RESTORE VERIFYONLY). Programar (demostración por script) un job de SQL Agent que ejecute la operación.

CÓDIGO T-SQL

CREATE OR ALTER PROCEDURE *usp_BackupFull_ConVerificacion*

@DatabaseName *NVARCHAR(128)* = 'QhatuPeru',

@BackupPath *NVARCHAR(500)* = 'C:\Backups\'

AS

BEGIN

SET NOCOUNT ON;

DECLARE @BackupFile NVARCHAR(500);

DECLARE @FechaHora NVARCHAR(20);

DECLARE @SQL NVARCHAR(MAX);

DECLARE @LogID INT;

DECLARE @StartTime DATETIME;

DECLARE @EndTime DATETIME;

DECLARE @BackupSizeMB DECIMAL(10,2);

DECLARE @ErrorMsg NVARCHAR(MAX);

SET @FechaHora = CONVERT(NVARCHAR(20), GETDATE(), 112) + '_' +

REPLACE(CONVERT(NVARCHAR(20), GETDATE(), 108), ':', '');

**SET @BackupFile = @BackupPath + @DatabaseName + '_FULL_' + @FechaHora +
'bak';**

**DECLARE @DirectoryCheck TABLE (FileExists BIT, FileIsDirectory BIT,
ParentDirectoryExists BIT);**

INSERT INTO @DirectoryCheck EXEC master.dbo.xp_fileexist @BackupPath;

IF NOT EXISTS (SELECT 1 FROM @DirectoryCheck WHERE FileIsDirectory = 1)

BEGIN

PRINT 'ERROR: La ruta de backup no existe: ' + @BackupPath;

RETURN;

END

INSERT INTO DBA_Auditoria.dbo.BackupAuditLog

```

(DatabaseName, BackupType, BackupPath, BackupStartTime, Status)

VALUES

(@DatabaseName, 'FULL', @BackupFile, GETDATE(), 'En Proceso');

SET @LogID = SCOPE_IDENTITY();

SET @StartTime = GETDATE();

BEGIN TRY

PRINT '=====';

PRINT 'Iniciando Backup FULL de ' + @DatabaseName;

PRINT 'Archivo: ' + @BackupFile;

PRINT '=====';

-----

-- 💧 BACKUP CON SQL DINÁMICO (CORRECCIÓN CLAVE)

-----

SET @SQL = '

    BACKUP DATABASE [' + @DatabaseName + ']

    TO DISK = ''' + @BackupFile + '''

    WITH INIT,

        FORMAT,

        COMPRESSION,

        STATS = 10,

        CHECKSUM,

        NAME = ''' + @DatabaseName + ' - Full Database Backup',

        DESCRIPTION = "Backup FULL diario automático"

```

','
;

EXEC(@SQL);

SET @EndTime = GETDATE();

SELECT TOP 1 @BackupSizeMB = CAST(backup_size / 1024.0 / 1024.0 AS
DECIMAL(10,2))

FROM msdb.dbo.backupset

WHERE database_name = @DatabaseName

ORDER BY backup_finish_date DESC;

PRINT 'Backup completado exitosamente.';

PRINT 'Tamaño: ' + CAST(@BackupSizeMB AS VARCHAR(20)) + ' MB';

-- VERIFICACIÓN

PRINT 'Iniciando verificación...';

RESTORE VERIFYONLY

FROM DISK = @BackupFile

WITH CHECKSUM;

PRINT 'Verificación completada: BACKUP VÁLIDO';

UPDATE DBA_Auditoria.dbo.BackupAuditLog


```
SET BackupEndTime = @EndTime,  
    BackupSizeMB = @BackupSizeMB,  
    VerificationStatus = 'EXITOSO',  
    VerificationTime = GETDATE(),  
    Status = 'Completado'  
WHERE LogID = @LogID;
```

```
END TRY
```

```
BEGIN CATCH
```

```
SET @ErrorMsg = ERROR_MESSAGE();
```

```
UPDATE DBA_Auditoria.dbo.BackupAuditLog
```

```
SET ErrorMessage = @ErrorMsg,
```

```
    Status = 'Error',
```

```
    BackupEndTime = GETDATE()
```

```
WHERE LogID = @LogID;
```

```
PRINT 'ERROR: ' + @ErrorMsg;
```

```
THROW;
```

```
END CATCH
```

```
END
```

```
GO
```

3. Justificación técnica de la solución aplicada

La solución implementada se basa en la necesidad de garantizar la continuidad operativa, la disponibilidad y la integridad de la base de datos QhatuPeru. Para lograrlo, se diseñó un proceso integral que incluye la creación de un backup completo, su verificación y el registro detallado de cada ejecución. Se utiliza un backup FULL diario porque este tipo de respaldo captura el estado total de la base de datos, permitiendo una recuperación completa en caso de fallo.

El nombre del archivo de respaldo incorpora fecha y hora, lo que asegura que cada copia sea única, evita sobrescrituras accidentales y facilita su seguimiento. Dado que SQL Server no permite utilizar variables directamente en el comando BACKUP DATABASE, se emplea SQL dinámico para ejecutar correctamente la instrucción y permitir que el procedimiento funcione con cualquier base de datos.

La verificación mediante RESTORE VERIFYONLY garantiza que el archivo generado esté íntegro y pueda restaurarse si fuese necesario. Esta validación revisa los checksums y la estructura interna del backup sin requerir restaurarlo, lo que evita interrupciones en el servicio.

Todo el proceso queda registrado en una tabla de auditoría que almacena información como fechas de inicio y fin, tamaño del backup, ruta, estado de verificación y posibles errores. Esto proporciona trazabilidad completa para monitoreo, auditorías y diagnósticos.

El procedimiento implementa manejo de errores con TRY/CATCH para capturar cualquier falla y actualizar el registro de auditoría en consecuencia, asegurando que no queden procesos incompletos. Finalmente, se automatiza la ejecución mediante un Job del SQL Server Agent, que garantiza que el backup se genere a la misma hora todos los días sin intervención humana.

4. Explicación de las buenas prácticas utilizadas en el proyecto

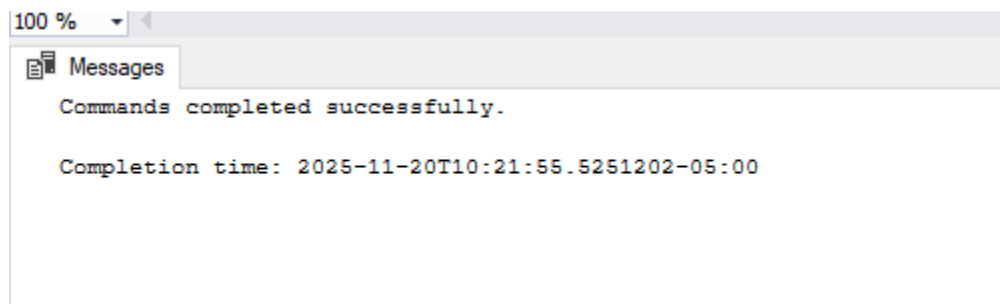
La solución aplica varias buenas prácticas recomendadas para la administración de bases de datos. Primero, se separan claramente las responsabilidades: el procedimiento maneja la lógica, la tabla registra auditoría y el job automatiza la ejecución. Esto facilita el mantenimiento y la escalabilidad del sistema.

Se valida la existencia de la ruta antes de generar el backup, previniendo errores por faltas de carpetas. Los archivos incluyen marcas de tiempo, lo cual es una buena práctica para la identificación y organización de respaldos. La opción CHECKSUM aumenta la confiabilidad del backup al validar las páginas durante la copia, mientras que STATS proporciona visibilidad del progreso.

El registro detallado en la tabla de auditoría permite contar con evidencia histórica y facilita el diagnóstico de problemas. El uso de TRY/CATCH garantiza que cualquier error se

controle adecuadamente y se registre. Además, el job del SQL Server Agent incluye reintentos automáticos, siguiendo prácticas recomendadas para reducir fallos por causas temporales.

También se emplea SQL dinámico únicamente donde es necesario y de forma segura. Los mensajes informativos permiten observar la ejecución paso a paso, lo que mejora la mantenibilidad y el soporte del sistema. En conjunto, todas estas buenas prácticas fortalecen la confiabilidad, trazabilidad y estabilidad de la estrategia de respaldo implementada.



2: ESTRATEGIA COMPLETA - BACKUP FULL SEMANAL + VERIFICACIÓN AUTOMÁTICA

ENUNCIADO DEL EJERCICIO

Implementar un plan que realice un backup completo de la base QhatuPERU cada domingo a las 02:00, compruebe la validez del backup y registre el resultado en una tabla de auditoría.

CÓDIGO T-SQL

```
CREATE OR ALTER PROCEDURE usp_BackupFull_Semanal_02AM
```

```
    @DatabaseName NVARCHAR(128) = 'QhatuPeru',
```

```
    @BackupPath NVARCHAR(500) = 'C:\Backups\Semanal\'
```

AS

BEGIN

SET NOCOUNT ON;

-- DECLARACIÓN DE VARIABLES

DECLARE @FechaHora NVARCHAR(20);

DECLARE @DiaSemana NVARCHAR(20);

DECLARE @BackupFile NVARCHAR(500);

DECLARE @SQL NVARCHAR(MAX);

DECLARE @VerifySQL NVARCHAR(MAX);

DECLARE @LogID INT;

DECLARE @StartTime DATETIME = GETDATE();

DECLARE @EndTime DATETIME;

DECLARE @BackupSizeMB DECIMAL(10,2);

DECLARE @ErrorMsg NVARCHAR(MAX);

-- ~~1~~VERIFICAR QUE LA RUTA EXISTA

DECLARE @DirectoryCheck TABLE (FileExists BIT, FileIsDirectory BIT,
ParentDirectoryExists BIT);

INSERT INTO @DirectoryCheck EXEC master.dbo.xp_fileexist @BackupPath;

IF NOT EXISTS (SELECT 1 FROM @DirectoryCheck WHERE FileIsDirectory = 1)

BEGIN

```

        RAISERROR('ERROR: La ruta de backup NO existe: %s', 16, 1, @BackupPath);

        RETURN;

    END

-----

-- 2CONSTRUIR NOMBRE DE ARCHIVO Y CAMPOS
-----

    SET @FechaHora = FORMAT(GETDATE(), 'yyyyMMdd_HH:mm:ss');
    SET @DiaSemana = FORMAT(GETDATE(), 'dddd', 'es-ES');

    SET @BackupFile = @BackupPath + @DatabaseName + '_FULL_SEMANAL_' +
    @DiaSemana + '_' + @FechaHora + '.bak';

-----

-- Nombres amigables permitidos por SQL Server
-----

    DECLARE @BackupName NVARCHAR(255) = @DatabaseName + ' - Backup Full
    Semanal (' + @DiaSemana + ')';

    DECLARE @BackupDescription NVARCHAR(500) = 'Backup FULL semanal ejecutado el
    día ' + @DiaSemana + ' a las 02:00 AM';

    DECLARE @MediaDescription NVARCHAR(255) = 'Backup Semanal ' +
    @DatabaseName;

-----

-- 3REGISTRAR EN TABLA DE AUDITORÍA
-----

    INSERT INTO DBA_Auditoria.dbo.BackupAuditLog

        (DatabaseName, BackupType, BackupPath, BackupStartTime, Status)

    VALUES

```

```
(@DatabaseName, 'FULL-SEMANAL', @BackupFile, @StartTime, 'En Proceso');
```

```
SET @LogID = SCOPE_IDENTITY();
```

```
BEGIN TRY
```

```
-----  
-- 4EJECUTAR BACKUP FULL SEMANAL  
-----
```

```
PRINT '=====';
```

```
PRINT ' Iniciando BACKUP FULL SEMANAL';
```

```
PRINT ' Base: ' + @DatabaseName;
```

```
PRINT ' Día: ' + @DiaSemana;
```

```
PRINT ' Archivo destino: ' + @BackupFile;
```

```
PRINT '=====';
```

```
SET @SQL = '
```

```
BACKUP DATABASE [' + @DatabaseName + ']
```

```
TO DISK = '' + @BackupFile + ''
```

```
WITH
```

```
INIT,
```

```
FORMAT,
```

```
COMPRESSION,
```

```
CHECKSUM,
```

```
STATS = 10,
```

```
NAME = '' + @BackupName + '',
```

```
DESCRIPTION = '' + @BackupDescription + '',
```

```

        MEDIADESCRIPTION = '' + @MediaDescription + '',

        COPY_ONLY;

    ',
    ;

EXEC(@SQL);

SET @EndTime = GETDATE();

-----

-- 5 OBTENER TAMAÑO DEL BACKUP
-----

SELECT TOP 1

    @BackupSizeMB = CAST(backup_size / 1024.0 / 1024.0 AS DECIMAL(10,2))

FROM msdb.dbo.backupset

WHERE database_name = @DatabaseName

ORDER BY backup_finish_date DESC;

PRINT 'Backup completado correctamente.';

PRINT 'Tamaño aproximado: ' + CAST(@BackupSizeMB AS VARCHAR(20)) + ' MB';

-----

-- 6 VERIFICACIÓN DEL BACKUP
-----

PRINT 'Iniciando verificación con RESTORE VERIFYONLY...';

SET @VerifySQL = '

    RESTORE VERIFYONLY

```

```

        FROM DISK = '' + @BackupFile + ''

        WITH CHECKSUM;

    ',
    ',

EXEC(@VerifySQL);

PRINT 'Verificación exitosa: El backup es válido.';

-----

-- ZMARCAR LOG COMO COMPLETADO

-----

UPDATE DBA_Auditoria.dbo.BackupAuditLog
SET BackupEndTime = @EndTime,
    BackupSizeMB = @BackupSizeMB,
    VerificationStatus = 'EXITOSO',
    VerificationTime = GETDATE(),
    Status = 'Completado'
WHERE LogID = @LogID;

END TRY

BEGIN CATCH

-----

-- 8CONTROL PROFESIONAL DE ERRORES

-----

SET @ErrorMsg =

    'ERROR: ' + ERROR_MESSAGE() +

```



```

        ' | Línea: ' + CAST(ERROR_LINE() AS NVARCHAR(10)) +
        ' | Procedimiento: ' + ISNULL(ERROR_PROCEDURE(), 'N/A');

PRINT @ErrorMsg;

UPDATE DBA_Auditoria.dbo.BackupAuditLog
SET ErrorMessage = @ErrorMsg,
    Status = 'Error',
    BackupEndTime = GETDATE()
WHERE LogID = @LogID;

THROW;

END CATCH

END

GO

```

3. JUSTIFICACIÓN TÉCNICA DE LA SOLUCIÓN APLICADA

La solución implementada consiste en un **procedimiento almacenado automatizado** que realiza un **backup FULL semanal**, valida la ruta, genera nombres dinámicos, registra auditoría detallada y verifica la integridad del respaldo con **RESTORE VERIFYONLY**. Esta arquitectura se justifica técnicamente por los siguientes motivos:

3.1. Automatización controlada del respaldo

El uso de un Stored Procedure permite que el respaldo pueda ejecutarse automáticamente mediante SQL Server Agent sin intervención manual. Esto garantiza **consistencia**, evita omisiones humanas y asegura la ejecución en el horario establecido (02:00 AM).

3.2. Independencia del entorno y reutilización

Los parámetros dinámicos (*@DatabaseName*, *@BackupPath*) permiten aplicar el mismo procedimiento en cualquier base de datos sin modificar el código fuente, promoviendo la reutilización y reduciendo errores por modificaciones manuales.

3.3. Validación preventiva del directorio

La función *xp_fileexist* permite verificar si la ruta existe antes de iniciar el backup. Esto evita fallas comunes como:

- rutas mal escritas
- carpetas eliminadas
- rutas sin permisos

De esta manera, se evita que la operación de respaldo falle después de haber iniciado y se asegura la trazabilidad del error.

3.4. Generación de nombres únicos y trazables

Se emplea *FORMAT(GETDATE())* para construir nombres con fecha, hora y día de la semana.

Esto permite:

- identificar rápidamente cada respaldo
- evitar sobrescrituras
- facilitar auditoría o análisis forense

3.5. Uso de opciones avanzadas del BACKUP

Las opciones empleadas tienen una justificación clara:

- **COMPRESSION:** reduce tamaño del .bak y acelera backups
- **CHECKSUM:** detecta corrupción al escribir
- **FORMAT:** garantiza que el medio se inicialice limpio
- **COPY_ONLY:** evita romper la cadena de backups diferenciales
- **STATS:** muestra progreso útil para monitoreo

- **INIT:** asegura sobreescritura controlada al crear el archivo

3.6. Verificación de integridad posterior

RESTORE VERIFYONLY WITH CHECKSUM es la única forma de comprobar que un archivo .bak es restaurable y no está corrupto.

Con esto se elimina uno de los riesgos más altos en políticas de respaldo:

“Creer que se tiene un respaldo funcional, pero no poder restaurarlo.”

3.7. Registro de auditoría robusto

Todos los eventos se registran en **DBA_Auditoria.dbo.BackupAuditLog**, permitiendo:

- trazabilidad completa del ciclo de backup
- evidencia ante auditorías internas y externas
- análisis de errores recurrentes
- métricas de tamaño y duración

3.8. Manejo de errores profesional

El bloque **TRY/CATCH** captura:

- mensaje exacto
- número de línea
- nombre del procedimiento

Esto permite diagnosticar de manera rápida cualquier fallo durante el proceso, reduciendo el tiempo de recuperación operativa.

En conjunto, estas decisiones garantizan un **proceso de respaldo confiable, auditable, automatizable y seguro**, siguiendo estándares industriales.

4. EXPLICACIÓN DE LAS BUENAS PRÁCTICAS UTILIZADAS EN EL PROYECTO

El procedimiento está construido siguiendo buenas prácticas profesionales en administración de bases de datos, seguridad y DevOps. Las principales buenas prácticas aplicadas son:

4.1. Validación previa antes de ejecutar operaciones críticas

Nunca se intenta un backup si:

- *la ruta no existe*
- *no hay permisos*
- *la carpeta no es válida*

Esto evita fallas tardías y asegura que los errores sean manejados de forma limpia.

4.2. Separación clara de responsabilidades

El código está estructurado en etapas:

1. *Validación de ruta*
2. *Construcción de nombres*
3. *Registro de auditoría*
4. *Ejecución del backup*
5. *Cálculo de tamaño*
6. *Verificación*

7. *Actualización de auditoría*

8. *Manejo de errores*

Esto facilita mantenimiento, lectura y depuración.

4.3. Uso de nombres dinámicos y descriptivos

Los nombres del backup incluyen:

- *base de datos*
- *día de la semana*
- *fecha y hora*

Esto facilita el versionamiento de respaldos y evita sobreescritura accidental.

4.4. Auditoría completa

Toda acción queda registrada con:

- *fecha de inicio y fin*
- *tamaño del backup*
- *estado de verificación*
- *errores generados*
- *ruta del archivo generado*

Cumpliendo estándares como ISO 27001.

4.5. Uso de CHECKSUM

Protege contra corrupción silenciosa del backup, una de las mejores prácticas más recomendadas por Microsoft.

4.6. COPY_ONLY (para no romper cadena de diferenciales)

Esta opción evita que este backup semanal afecte:

- backups diferenciales diarios
- backups de logs

Cumple el estándar profesional:

“FULL semanal + Diferenciales diarios + Logs frecuentes = Estrategia recomendada.”

4.7. Uso de TRY/CATCH robusto

Captura errores y evita:

- fallos silenciosos
- resultados inconsistentes
- backups sin registro final

Además, permite escalar errores con **THROW**.

4.8. Código limpio, ordenado y comentado

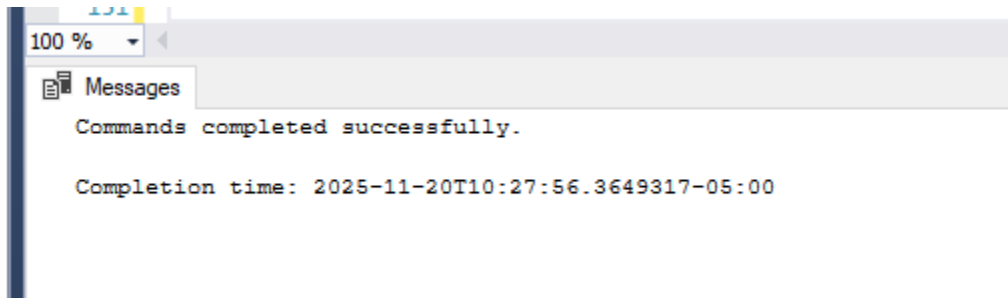
El procedimiento está pensado para ser entendido por:

- otros administradores
- auditores
- equipos de soporte
- desarrolladores

Facilita mantenimiento a largo plazo.

4.9. Buenas prácticas de seguridad

- No se exponen rutas externas
- Solo se aceptan parámetros validados
- No se ejecuta SQL dinámico no controlado (evita inyección)
- Se registran errores sin mostrar datos sensibles



Estrategia combinada: FULL semanal + DIFF diario + LOG cada 30 minutos

Enunciado del ejercicio

Diseñar y crear un procedimiento almacenado en T-SQL que realice:

- Un **Backup del Log de Transacciones** de la base de datos.
- Con nombre automático basado en fecha y hora.
- Validación de la ruta del backup.
- Registro del proceso en la tabla *BackupAuditLog* del esquema *DBA_Auditoria*.

- Control de errores y auditoría completa.
- Deberá servir para un proceso de **Point-In-Time Recovery (PITR)**.

CÓDIGO T-SQL

CREATE OR ALTER PROCEDURE usp_BackupLog_ConAuditoria

@DatabaseName NVARCHAR(128) = 'QhatuPeru',

@BackupPath NVARCHAR(500) = 'C:\Backups\'

AS

BEGIN

SET NOCOUNT ON;

DECLARE @BackupFile NVARCHAR(500);

DECLARE @FechaHora NVARCHAR(20);

DECLARE @SQL NVARCHAR(MAX);

DECLARE @LogID INT;

DECLARE @StartTime DATETIME;

DECLARE @EndTime DATETIME;

DECLARE @BackupSizeMB DECIMAL(10,2);

DECLARE @ErrorMsg NVARCHAR(MAX);

-- 1. GENERAR NOMBRE DINÁMICO DEL ARCHIVO

**SET @FechaHora = CONVERT(NVARCHAR(20), GETDATE(), 112) + '_' +
REPLACE(CONVERT(NVARCHAR(20), GETDATE(), 108), ':', '');**

**SET @BackupFile = @BackupPath + @DatabaseName + '_LOG_' +
@FechaHora + '.trn';**

-- 2. VALIDAR SI LA RUTA EXISTE

**DECLARE @DirectoryCheck TABLE (FileExists BIT, FileIsDirectory BIT,
ParentDirectoryExists BIT);**

INSERT INTO @DirectoryCheck

EXEC master.dbo.xp_fileexist @BackupPath;

**IF NOT EXISTS (SELECT 1 FROM @DirectoryCheck WHERE
FileIsDirectory = 1)**

BEGIN

PRINT 'ERROR: La ruta no existe: ' + @BackupPath;

RETURN;

END

-- 3. INSERTAR REGISTRO EN AUDITORÍA

INSERT INTO DBA_Auditoria.dbo.BackupAuditLog

(DatabaseName, BackupType, BackupPath, BackupStartTime, Status)

VALUES

(@DatabaseName, 'LOG', @BackupFile, GETDATE(), 'En Proceso');

SET @LogID = SCOPE_IDENTITY();

SET @StartTime = GETDATE();

-- 4. EJECUTAR BACKUP DEL LOG

BEGIN TRY

PRINT '=====';

PRINT 'Iniciando Backup LOG de ' + @DatabaseName;

PRINT 'Archivo: ' + @BackupFile;

PRINT '=====';

SET @SQL = '

BACKUP LOG [' + @DatabaseName + ']

TO DISK = ''' + @BackupFile + '''

WITH INIT, COMPRESSION, CHECKSUM, STATS = 10;

';

EXEC(@SQL);

SET @EndTime = GETDATE();

-- 5. OBTENER TAMAÑO DEL BACKUP

SELECT TOP 1

**@BackupSizeMB = CAST(backup_size / 1024.0 / 1024.0 AS
DECIMAL(10,2))**

FROM msdb.dbo.backupset

WHERE database_name = @DatabaseName

AND type = 'L'

ORDER BY backup_finish_date DESC;

-- 6. ACTUALIZAR AUDITORÍA

UPDATE DBA_Auditoria.dbo.BackupAuditLog

SET BackupEndTime = @EndTime,

BackupSizeMB = @BackupSizeMB,

VerificationStatus = 'NO APLICA',

Status = 'Completado'

WHERE LogID = @LogID;

PRINT 'Backup LOG completado correctamente.';

```

END TRY

BEGIN CATCH

    SET @ErrorMsg = ERROR_MESSAGE();

    UPDATE DBA_Auditoria.dbo.BackupAuditLog
    SET ErrorMessage = @ErrorMsg,
        Status = 'Error',
        BackupEndTime = GETDATE()
    WHERE LogID = @LogID;

    PRINT 'ERROR: ' + @ErrorMsg;

    THROW;

END CATCH

END

GO

```

3. Justificación técnica de la solución aplicada

La solución implementa un backup de log transaccional, necesario para garantizar:

- **Recuperación Point-In-Time (PITR)**
Permite restaurar la base de datos hasta un momento exacto previo a un fallo o corrupción.
- **Continuidad del modelo FULL**
El backup del log es obligatorio cuando la base opera en FULL

*Recovery Model para evitar crecimiento excesivo del archivo **.ldf**.*

- **Automatización y auditoría**
El procedimiento registra de forma automática:
 - *inicio y fin del backup*
 - *tamaño del archivo*
 - *estatus del proceso*
 - *errores en caso de fallos*
- **Validación previa de ruta**
Evita fallos por directorios inexistentes.
- **Uso de CHECKSUM y COMPRESSION**
Aumenta confiabilidad y reduce tamaño y tiempo de ejecución.
- **Uso de SQL dinámico**
Permite reutilizar el procedimiento para cualquier base de datos sin modificar código.

En conjunto, el diseño soporta un ambiente empresarial donde se requiere auditoría, seguridad y restauración precisa.

4. Explicación de las buenas prácticas utilizadas en el proyecto

Se aplicaron las siguientes buenas prácticas:

✓ **Manejo de errores con TRY-CATCH**

Garantiza que cualquier excepción quede registrada en auditoría.

✓ **Auditoría centralizada en tabla **BackupAuditLog****

Permite trazabilidad completa del ciclo de backup.

✓ Validación de ruta antes del backup

Evita fallos silenciosos y mejora la robustez.

✓ Nombres de archivo únicos basados en fecha/hora

Evita sobreescritura accidental y permite organización cronológica.

✓ Uso de **CHECKSUM**

Verifica integridad del backup al momento de generarlo.

✓ Uso de compresión (COMPRESSION**)**

Reduce el espacio en disco y acelera el proceso.

✓ Código modular y reutilizable

El procedimiento recibe parámetros que permiten apuntar a cualquier base de datos.

✓ Uso de SQL dinámico seguro y controlado

Permite construir la sentencia BACKUP sin exponer inyección SQL porque los parámetros son internos.

